

CS 486/686: Assignment 3 (100 Points)

Instructors: Yuntian Deng

Due Date: Tuesday, August 4, 2026 at 11:59 pm (ET; Waterloo Time)

Instructions

- **Warning:** This assignment is longer and more involved than previous assignments. Start early to ensure timely completion.
- Submit your solutions to the Assignment 3 Dropbox on LEARN. Your submission must include:
 - A file named `wakeup.pdf` that contains all your written solutions (Question 1, Question 2, and, if attempted, the Bonus Question);
 - A file named `code.py` that contains your code for Question 1;
 - Your completed `neural_net.py` and `operations.py` for Question 2;
 - (Optional) A file named `train_model_pytorch.py` for the Bonus Question.

If your submission does not contain `wakeup.pdf`, `code.py` (Question 1), and your completed `neural_net.py` and `operations.py` (Question 2), we will deduct 5 points from your final assignment mark.

- Upload each file directly to the dropbox, using the exact file names listed above. Do not put them inside a folder or a zip archive (the Question 2 files are imported by name during grading).
- Submit only the files listed above. Do not include the provided data or test files (e.g., the datasets or the tests/ folder).
- Use Python 3.12 or newer with NumPy installed to implement and run the code.
- No late assignment will be accepted.
- This assignment is to be done individually.

1 Question 1 [35pts]: Text Categorization with Decision Trees

Text categorization is an important task in natural language processing and information retrieval. For instance, news articles, emails or blogs are often classified by topics. In this assignment, you will implement, in Python, a decision tree algorithm to learn a classifier that can assign a discussion board topic to an article.

Train and test your algorithm with the datasets `trainData.txt` and `testData.txt` provided in the assignment bundle on LEARN. The dataset includes the first 1500 comments of August 2019 of each of the `r/books` and `r/atheism` subreddits, cleaned by removing punctuation and some offensive language, and limiting the words to only those used more than 3 times among all posts. These 3000 comments are split evenly into training and testing sets (with 1500 documents in each). To simplify your implementation, these posts have been pre-processed and converted to the *bag of words* model. More precisely, each post is converted to a vector of binary values such that each entry indicates whether the document contains a specific word or not.

Each line of the files `trainData.txt` and `testData.txt` are formatted as “`docId wordId`”, which indicates that word `wordId` is present in document `docId`. Files `trainLabel.txt` and `testLabel.txt` indicate the label/category (1=atheism or 2=books) for each document (`docId` = line#). The file `words.txt` indicates which word corresponds to each `wordId` (denoted by the line#).

Implement a decision tree learning algorithm. Here, each decision node corresponds to a word feature, and the leaf nodes correspond to a prediction of what subreddit the article belongs in. You will experiment with two feature selection mechanisms as follows:

1. Average information gain (evenly weighted across the leaves)

$$I_G = I(E) - \left[\frac{1}{2}I(E_1) + \frac{1}{2}I(E_2) \right];$$

2. Information gain weighted by the fraction of documents on each side of the split

$$I_G = I(E) - \left[\frac{N_1}{N}I(E_1) + \frac{N_2}{N}I(E_2) \right],$$

where $I(E)$ is the information content (entropy) in the N examples before the split, and $I(E_i)$ is the information content of the N_i examples in the i^{th} leaf after the split. Use $\log_2$ when computing entropy, so that the entropy of a balanced two-class set is 1. Method (1) does not deal well with splits that put a lot of examples on one side, and very few on the other. For example, suppose you have evenly distributed data across the two target classes (so $I(E) = 1$): N examples, and 2 features. Suppose the first feature splits one example off from the other

$N - 1$, so Method (2) gives:

$$\begin{aligned} I(E_1) &= 0, \\ I(E_2) &\approx 1, \\ I_G &\approx 1 - \left[\frac{1}{N} \times 0 + \frac{N-1}{N} \times 1 \right] = \frac{1}{N}. \end{aligned}$$

That is, using Method (2), I_G is a small value. On the other hand, Method (1) will make I_G look better than it should be, since it will compute:

$$I_G \approx 1 - \left[\frac{1}{2} \times 0 + \frac{1}{2} \times 1 \right] = \frac{1}{2}.$$

(Method (2), the fraction-weighted gain, is the version we used in lecture; Method (1) is included here so you can compare the two feature-scoring criteria.)

Entropy of an empty subset. A split can send all of a leaf's examples to one child and none to the other (this happens when a word is present in every document at the leaf, or in none of them). The empty child then has no examples, and entropy is only defined for an actual class distribution, so we must adopt a convention for $I(\{\})$. For this assignment, **assume the entropy of an empty subset is maximal**, i.e., use $I(\{\}) = 1$.

Why maximal (1) rather than 0? Consider such a degenerate split, where $E_1 = E$ (all examples) and $E_2 = \{\}$ (empty). Under Method (1), setting $I(\{\}) = 0$ would give $I_G = I(E) - \left[\frac{1}{2}I(E) + \frac{1}{2} \cdot 0 \right] = \frac{1}{2}I(E) > 0$, so a split that separates nothing would look beneficial. Setting $I(\{\}) = 1$ instead gives $I_G = \frac{1}{2}(I(E) - 1) \leq 0$, correctly indicating no useful split. This is the same failure mode of Method (1) illustrated above. Under Method (2) the value of $I(\{\})$ does not actually matter, because an empty child has weight $\frac{N_i}{N} = 0$, so its term vanishes regardless. Note that an empty subset (no examples, $I = 1$ by convention) is different from a pure leaf (all examples in one class), whose entropy is genuinely 0.

Build each decision tree by maintaining a priority queue of leaves, where each leaf is prioritized by the best information gain achievable by splitting it (i.e., the information gain of the single best word feature to split that leaf on, under the chosen measure). At each iteration, expand the leaf at the front of the queue by splitting it on that best word feature. Grow the tree one node at a time, up to 100 internal nodes, training on the training set only. The prediction at each leaf should be the majority class of the training examples that reach it (not a probability).

To keep results reproducible and gradeable, break ties deterministically: when two word features give the same information gain, choose the one with the smaller wordId; when two leaves have the same best gain, expand the one that was created earlier.

Plot the training and testing accuracy (i.e., percentage of correctly classified articles) after adding each new node. Generate two graphs (one for each method of feature selection) and

plot two curves on each (one for training accuracy and one for test accuracy). These graphs should have accuracy as a percentage on the y-axis and number of nodes (1-100) on the x-axis. You may generate them however you wish (e.g. Matplotlib or Seaborn for Python).

What to hand in:

1. **[15 pts]** Your code, which must be your own, and you must use the iterative priority queue model. If your code is recursive, or doesn't expand nodes by order of information gain throughout the tree, you will not get these pts.
2. **[10 pts]** A printout (or hand drawing) showing two decision trees (one tree for each feature selection method) after adding the first 10 word features. Show the (method-specific) information gain and word feature selected at each internal node. Clearly indicate which branch corresponds to the presence (solid line) or absence (dashed line) of the word. Show the prediction at each leaf. Each printed or drawn tree should have 10 internal nodes and 11 leaves.
3. **[10 pts]** Two graphs (one for each feature selection method) showing the training and testing accuracy as the number of nodes increases from 1 to 100.

2 Question 2 [65pts]: Neural Networks

In this part of the assignment, you will implement a feedforward neural network from scratch. Additionally, you will implement activation functions, a loss function, and a performance metric. Lastly, you will train a neural network model to perform a regression problem.

Red Wine Quality - A Regression Problem

The task is to predict the quality of red wine from northern Portugal, given some physical characteristics of the wine. The target $y \in [0, 10]$ is a continuous variable, where 10 is the best possible wine, according to human tasters. This dataset was downloaded from the UCI Machine Learning Repository.

The features are all real-valued. They are listed below:

- | | | |
|--------------------|------------------------|-------------|
| • Fixed acidity | • Chlorides | • pH |
| • Volatile acidity | • Free sulfur dioxide | • Sulphates |
| • Citric acid | • Total sulfur dioxide | |
| • Residual sugar | • Density | • Alcohol |

Training a Neural Network

In this assignment, you will apply the forward and backward pass to the entire dataset simultaneously (i.e. batch gradient descent). As a result, your forward and backward passes will manipulate tensors, where the first dimension is the number of examples in the training set, n . Because the loss $\mathcal{L}$ is a mean over the n examples (it already contains a $\frac{1}{n}$ factor), the gradient $\frac{\partial \mathcal{L}}{\partial W_{i,j}^{(l)}}$ for each weight is obtained by summing each example's contribution to that mean; do not divide by n a second time. Algorithm 1 gives the training algorithm in terms of functions that you will implement in this assignment. Further details can be found in the documentation for each function in the provided source code.

Notation. We follow the same convention as lecture. For each layer l , let $z^{(l)}$ be the **pre-activation** (the weighted sum) and $a^{(l)} = g(z^{(l)})$ be the **activation** (the output after applying the activation function g). The input is $a^{(0)} = X$, and the network's prediction is $\hat{Y} = a^{(L)}$ (the activation of the last layer). In code these correspond to the two lists returned by `forward_pass`: `z_vals` holds the pre-activations $z^{(l)}$ and `a_vals` holds the activations $a^{(l)}$. Each weight matrix $W^{(l)}$ includes an extra first row for the **bias**: the forward pass augments the incoming activations with a constant-1 column, so $z^{(l)} = [\mathbf{1} \ a^{(l-1)}] W^{(l)}$. The backward pass computes, for each layer, the **delta** $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial z^{(l)}}$ (the gradient of the loss with respect to that layer's pre-activation); `update_weights` then turns these deltas into the weight gradients.

Algorithm 1 Training

Require: $\eta > 0$ ▷ Learning rate
Require: $n_{\text{epochs}} \in \mathbb{N}^+$ ▷ Number of epochs
Require: $X \in \mathbb{R}^{n \times f}$ ▷ Training examples with n examples and f features
Require: $y \in \mathbb{R}^n$ ▷ Targets for training examples
 Initiate weight matrices $W^{(l)}$ randomly for each layer. ▷ Initialize net
for $i \in \{1, 2, \dots, n_{\text{epochs}}\}$ **do** ▷ Conduct n_{epochs} epochs
 $z_vals, a_vals \leftarrow \text{net.forward_pass}(X)$ ▷ Forward pass
 $\hat{Y} \leftarrow a_vals[-1]$ ▷ Predictions
 $L \leftarrow \mathcal{L}(\hat{Y}, Y)$
 Compute $\frac{\partial}{\partial \hat{Y}} \mathcal{L}(\hat{Y}, Y)$ ▷ Derivative of error with respect to predictions
 $deltas \leftarrow \text{backward_pass}(z_vals, \frac{\partial}{\partial \hat{Y}} \mathcal{L}(\hat{Y}, Y))$ ▷ Backward pass
 $\text{update_gradients}()$ ▷ $W_{i,j}^{(\ell)} \leftarrow W_{i,j}^{(\ell)} - \eta \sum_n \frac{\partial \mathcal{L}}{\partial W_{i,j}^{(\ell)}}$ for each weight
end for
return trained weight matrices $W^{(\ell)}$

Activation and Loss Functions

You will implement the following activation functions and their derivatives:

- **Sigmoid**

$$g(x) = \frac{1}{1 + e^{-kx}}$$

- **ReLU**

$$g(x) = \max(0, x)$$

You will derive and implement each activation's derivative yourself. Two conventions the autograder relies on: (i) the sigmoid derivative depends on the steepness parameter k (it does not cancel out); and (ii) ReLU is not differentiable at 0, so use the convention $g'(0) = 0$.

You will implement the following loss function and its derivative for regression:

- **Mean Squared Error**

$$\mathcal{L}(\hat{Y}, Y) = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

When you differentiate the mean squared error, remember that it is averaged over the n examples, so the $\frac{1}{n}$ factor carries through into $\frac{\partial \mathcal{L}}{\partial \hat{Y}}$.

To evaluate the performance of the model, you will implement the following performance metric:

- **Mean Absolute Error**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{Y}_i - Y_i|$$

Implementation

We have provided three Python files. Please read the detailed comments in the provided files carefully. Note that some functions have already been implemented for you.

1. `neural_net.py`: Contains an implementation of a `NeuralNetwork` class. You must implement the `forward_pass()`, `backward_pass()`, and `update_weights()` methods in the `NeuralNetwork` class. **Do not change the function signatures. Do not change anything else in this file!**

2. `operations.py`: Contains classes for activation functions, a loss function, and a performance metric. The activation functions extend a base `Activation` class and the loss function extends a base `Loss` class. You must implement all the blank functions as indicated in this file. **Do not change the function signatures. Do not change anything else in this file!**
3. `train_experiment.py`: Provides a demonstration of how to define a `NeuralNetwork` object and train it on one of the provided datasets. Feel free to change this file as you desire.

Please complete the following tasks:

1. **[48 Points]** Implement the empty functions in `neural_net.py` and `operations.py`.

Please do not invoke any numpy random operations in `neural_net.py` and `operations.py`. This may throw the automatic grading off.

Unit tests for `neural_net.py`:

- `NeuralNetwork.forward_pass()`
(1 public test + 2 secret tests) * 3 marks = 9 marks
- `NeuralNetwork.backward_pass()`
(1 public test + 2 secret tests) * 3 marks = 9 marks
- `NeuralNetwork.update_weights()`
(1 public test + 2 secret tests) * 3 marks = 9 marks

Unit tests for `operations.py`:

- `Sigmoid.value()`
(1 public test + 2 secret tests) * 1 mark = 3 marks
- `Sigmoid.derivative()`
(1 public test + 2 secret tests) * 1 mark = 3 marks
- `ReLU.value()`
(1 public test + 2 secret tests) * 1 mark = 3 marks
- `ReLU.derivative()`
(1 public test + 2 secret tests) * 1 mark = 3 marks
- `MeanSquaredError.value()`
(1 public test + 2 secret tests) * 1 mark = 3 marks
- `MeanSquaredError.derivative()`
(1 public test + 2 secret tests) * 1 mark = 3 marks
- `mean_absolute_error`
(1 public test + 2 secret tests) * 1 mark = 3 marks

We have included the public tests in the provided Question 2 bundle (a3_q2.zip) under `tests/`, where `nets.json` specifies the initial network weights used for the forward, backward, and weight-updating tests. Run `python run_public_tests.py` to check your implementation against the public tests (at least on public tests). The secret tests will be used for grading.

Once you have implemented the functions, you can try training neural networks on the provided dataset. The wine quality dataset is in a3_q2.zip under `data/wine_quality.csv`.

In `train_experiment.py`, we have provided some code to instantiate a neural network and train on an entire dataset. Experiment with different numbers of layers with different sizes and activation functions. You do not need to submit the results of this experimentation.

2. [17 points] Execute k -fold cross validation for the wine quality dataset with $k = 5$.

What is k -fold cross validation? It is a way to estimate how well a model generalizes when data is limited. Partition the dataset into k equal-sized parts (“folds”). Then run k experiments: in experiment i , hold out fold i as the validation set and train on the other $k - 1$ folds, then measure the trained model’s error on the held-out fold. Every example is used for validation exactly once. Averaging the k validation errors gives a more stable estimate than a single train/validation split, and their standard deviation indicates how sensitive the estimate is to which data was held out.

Use a single node with the Identity activation function as your output layer. Other layers are your choice. Report the sizes of the layers that you used in your neural network, along with the activation functions you used for your hidden layers. Train for 500 epochs in each trial and use $\eta = 0.001$.

To perform cross validation, randomly split the data into 5 folds. For each fold, train the model on the remaining data and determine the trained model’s mean absolute error on the fold. You can use `NeuralNetwork.evaluate()` to determine the mean absolute error on the validation set (i.e. fold).

Produce a plot where the x -axis is the epoch number and the y -axis is the average training loss across all experiments for the current epoch. Report the average and standard deviation of the mean absolute error across all folds after training for 500 epochs.

For example, for your first fold, 80% of the examples should be in the training set and 20% of the examples should be in the validation set (i.e. fold 1). You will require the loss obtained after executing the forward pass for each of the 500 epochs. After your model has trained, use the trained model to calculate the mean absolute error on the validation set. This is one experiment. You will need to run this experiment 5 times in total, plotting the average loss at epoch i for each epoch.

3 Bonus Question (Optional) [40 extra pts]: Training with PyTorch

In this optional bonus question, you'll revisit the wine quality regression task from Question 2, but this time using PyTorch's automatic differentiation functionality (autograd). You'll define your neural network using PyTorch modules, automatically compute gradients with `loss.backward()`, and optimize parameters using PyTorch's built-in optimizers.

Task Description:

- Load the dataset from `wine_quality.csv` from Question 2.
- Randomly split your dataset into 90% training examples and 10% validation examples. When splitting data, set a random seed (e.g., `random_seed=42`) to ensure reproducibility.
- Define your neural network using PyTorch's `nn.Module`. You are free to choose the number and sizes of hidden layers and activation functions, but your final output should be a single neuron.
- Compute the loss using PyTorch's built-in `nn.MSELoss`.
- Use a PyTorch optimizer (e.g., `torch.optim.Adam`) to train your neural network for 500 epochs, with a learning rate of your choice.
- After each epoch, evaluate your model's performance on the validation set using Mean Absolute Error (MAE). Record these values to produce your plot.

Pseudocode: Below is high-level pseudocode illustrating the expected workflow:

```
# Import libraries (torch, pandas, sklearn, matplotlib)

# Use provided helper function from earlier question
X, y = load_dataset("wine_quality.csv", "quality")

# Split data into 90% train and 10% validation sets

# Convert data to PyTorch tensors

# Define your neural network model (using nn.Module)

# Define MSE loss

# Define an optimizer (e.g. Adam), pass in model.parameters()
```

```
validation_mae = []
for epoch in range(500):
    # Perform forward pass, compute loss

    # Use loss.backward() for gradients

    # use optimizer.step() and optimizer.zero_grad()

    # Evaluate your model on validation data, compute MAE,
    # store this validation MAE for later plotting

# Plot validation MAE vs. epoch number
```

What to hand in:

1. **[30 pts]** Your Python implementation in a file named `train_model_pytorch.py`. Your code should demonstrate the correct usage of PyTorch's autograd (`loss.backward()` and `optimizer.step()`).
2. **[10 pts]** A clearly labeled plot of your validation Mean Absolute Error (MAE) as a function of epochs (from epoch 1 to epoch 500). Include this plot directly within your `writeup.pdf`.

Important Notes:

- This bonus question is completely optional. Not attempting it will not negatively affect your assignment grade.
- Submissions that do not clearly use PyTorch's automatic differentiation methods will receive no credit.